Table des matières

Labo 03 — Charger la RAW 1
Avant de commencer 2
Étape 1 — Créer l'entrepôt, et charger deux fois le même extrait 2
Étape 2 — Écrire le chargeur qui ne prend que le neuf et marque son lot 3
Étape 3 — Retrouver dans la RAW ce qu'elle garde exprès 8
Étape 4 — Déposer le corpus tel quel, avec son catalogue 9
Étape 5 — Versionner le chargeur et le dépôt du corpus 11
Dépannage 11
Annexe — Sur Snowflake 12
Annexe — Sur Databricks 14

Labo 03 — Charger la RAW

Labo · Cours 3/12 — Formation Data Engineering

But : créer l'entrepôt, voir ce que coûte un chargement qui ne se souvient de rien, puis charger les 22 livraisons de Boréal dans la couche `RAW` par un chargeur qu'on relance sans rien doubler ; retrouver dans la `RAW` ce qu'elle garde exprès ; déposer la documentation dans le bronze ; garder les deux scripts dans le dépôt. Le pourquoi de chaque étape est dans `Theorie_03_DE.md`, section 6 et annexe A.

Deux fenêtres de l'Anaconda Prompt, environnement `formations` actif, depuis `Projet_Boreal` :

- la première pour le travail : le Bloc-notes, Python et Git ;
- la seconde pour l'interface web de DuckDB (`duckdb -ui`) : les requêtes sur l'entrepôt.

Liste de contrôle :

- ☐ Entrepôt créé : `plateforme\entrepot\boreal.duckdb`
- ☐ Un extrait chargé deux fois à la main : 854 lignes pour 427 commandes
- ☐ Verrou vu : le chargeur refuse l'entrepôt que l'interface tient ouvert
- ☐ `charger_raw.py` lancé : 1 572 fichiers et 513 857 lignes, puis « à jour » partout

© 2026 iAlexMG. Tous droits réservés.

- ☐ `RAW` inspectée : 22 tables, 609 en-têtes au lot `2023-03_bis`, les doublons gardés, le client effacé
- ☐ `deposer_corpus.py` lancé : 35 fichiers déposés, puis 0
- ☐ Les deux scripts validés dans le dépôt `01_de`, `git status` propre

Avant de commencer

Anaconda Prompt, depuis `Projet_Boreal` :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir 01_de\sql
REM Attendu : trois fichiers, bronze_historien.sql, bronze_telemetrie.sql et premier_regard.sql.
dir plateforme\lac\bronze
REM Attendu : deux dossiers, telemetrie et usine.
```

Les lignes `REM` sont des commentaires : l'Anaconda Prompt les ignore. Si un résultat diffère, finir d'abord le Labo 02.

Étape 1 — Créer l'entrepôt, et charger deux fois le même extrait

Théorie : `Theorie_03_DE.md`, annexe A.1.

1.1 Créer le dossier de l'entrepôt. Première Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir plateforme\entrepot
```

Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Boreal`, lancer l'interface.

```
duckdb -ui
```

Laisser cette fenêtre ouverte : la fermer arrête l'interface. Créer un carnet neuf (+ à côté de Notebooks) et le renommer « Labo 03 » (double-clic sur son titre, puis Entrée). Une requête par cellule, Ctrl+Entrée pour l'exécuter.

1.2 Créer l'entrepôt en l'attachant à l'interface.

```
-- Attendu : 0 rows returned. entrepot paraît sous Attached databases, et le fichier
-- plateforme\entrepot\boreal.duckdb existe désormais.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot;
```


© 2026 iAlexMG. Tous droits réservés.

1.3 Charger à la main les en-têtes de commande de décembre 2025, dans un schéma d'essai. La cellule n'affiche que le résultat de sa dernière requête, le contrôle.

```
-- Attendu : 427 lignes.
CREATE SCHEMA IF NOT EXISTS entrepot.essai;
CREATE OR REPLACE TABLE entrepot.essai.vbak AS
SELECT * FROM read_csv('sources/erp/extraits/2025-12/VBAK.csv', all_varchar = true, filename = true);
SELECT COUNT(*) AS lignes FROM entrepot.essai.vbak;
```

1.4 Double compte voulu : ajouter le même fichier une seconde fois, comme un traitement relancé par erreur.

```
-- Attendu : 854 lignes, 427 commandes. Aucune erreur.
INSERT INTO entrepot.essai.vbak
SELECT * FROM read_csv('sources/erp/extraits/2025-12/VBAK.csv', all_varchar = true, filename = true);
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes FROM entrepot.essai.vbak;
```

À retenir — un `INSERT` relancé double les lignes, sans la moindre erreur. Le `CREATE OR REPLACE` de 1.3 remplaçait : le relancer redonne 427. L'`INSERT` ajoute : chaque commande de décembre est maintenant là deux fois. Un chargeur qui ajoute doit donc se souvenir de ce qu'il a déjà chargé : c'est le journal des chargements de l'étape 2.

1.5 Retirer l'essai. L'entrepôt reste attaché : l'étape 2 en a besoin.

```
-- Attendu : 0 rows returned.
DROP SCHEMA entrepot.essai CASCADE;
```

Étape 2 — Écrire le chargeur qui ne prend que le neuf et marque son lot

Théorie : `Theorie_03_DE.md`, annexe A.2.

2.1 Créer le script. Première Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir 01_de\ingestion
notepad 01_de\ingestion\charger_raw.py
```

Le Bloc-notes propose de créer le fichier : accepter. Coller, enregistrer, fermer. C'est un fichier Python : il ne se tape ni dans l'interface ni dans la ligne de commande DuckDB.

```
# -*- coding: utf-8 -*-
"""charger_raw.py : les fichiers livrés par les sources de Boréal, chargés dans la couche RAW.

Lancement, depuis Projet_Boreal/01_de : python ingestion/charger_raw.py
"""
import hashlib
```


© 2026 iAlexMG. Tous droits réservés.

```python
import os
import sys
import time
from datetime import datetime, timezone
from pathlib import Path

import duckdb

# 01_de/ingestion/charger_raw.py : deux crans plus haut, le dossier du projet.
PROJET = Path(__file__).resolve().parents[2]
ENTREPOT = "plateforme/entrepot/boreal.duckdb"

# Une livraison : la table de la RAW, le motif de ses fichiers, leur forme, et d'où vient le lot.
#   dossier : le dossier du mois (2023-03, 2023-03_bis) ; jour : la date du nom de fichier ;
#   complet : un export livré en entier, qui remplace le précédent.
LIVRAISONS = [
    ("erp_t005u", "sources/erp/extraits/*/T005U.csv", "csv", "dossier"),
    ("erp_kna1", "sources/erp/extraits/*/KNA1.csv", "csv", "dossier"),
    ("erp_mara", "sources/erp/extraits/*/MARA.csv", "csv", "dossier"),
    ("erp_makt", "sources/erp/extraits/*/MAKT.csv", "csv", "dossier"),
    ("erp_vbak", "sources/erp/extraits/*/VBAK.csv", "csv", "dossier"),
    ("erp_vbap", "sources/erp/extraits/*/VBAP.csv", "csv", "dossier"),
    ("erp_cdhdr", "sources/erp/extraits/*/CDHDR.csv", "csv", "dossier"),
    ("erp_cdpos", "sources/erp/extraits/*/CDPOS.csv", "csv", "dossier"),
    ("erp_zserie", "sources/erp/extraits/*/ZSERIE.csv", "csv", "dossier"),
    ("garanties_clients", "sources/garanties/*/clients.csv", "csv", "dossier"),
    ("garanties_enregistrements", "sources/garanties/*/enregistrements.csv", "csv", "dossier"),
    ("garanties_visites", "sources/garanties/*/visites.csv", "csv", "dossier"),
    ("garanties_consentements", "sources/garanties/*/consentements.csv", "csv", "dossier"),
    ("garanties_reclamations", "sources/garanties/*/reclamations.json", "json", "dossier"),
    ("usine_actifs", "sources/usine/referentiel/actifs.csv", "csv", "complet"),
    ("usine_etiquettes", "sources/usine/referentiel/etiquettes.csv", "csv", "complet"),
    ("usine_bons_travail", "sources/usine/gmao/bons_travail.csv", "csv", "complet"),
    ("meteo_stations", "sources/meteo/stations.csv", "csv", "complet"),
    ("meteo_quotidien", "sources/meteo/quotidien.csv", "csv", "complet"),
    ("meteo_horaire", "sources/meteo/horaire_7060400.csv", "csv", "complet"),
    ("telemetrie_sorties", "sources/telemetrie/sorties_*.jsonl", "jsonl", "jour"),
    ("corpus_index", "sources/corpus/index.csv", "csv", "complet"),
]

# La requête qui lit une liste de fichiers, selon leur forme. Chacune rend la colonne filename.
LECTEURS = {
```


© 2026 iAlexMG. Tous droits réservés.

```
    "csv": "SELECT * FROM read_csv($fichiers, header = true, all_varchar = true, filename = true)",
    "json": ("SELECT json AS objet, filename "
             "FROM read_json_objects($fichiers, format = 'array', filename = true)"),
    "jsonl": ("SELECT sujet, recu_le, qos, message, filename "
              "FROM read_json($fichiers, format = 'newline_delimited', filename = true, "
              "columns = {'sujet': 'VARCHAR', 'recu_le': 'VARCHAR', 'qos': 'VARCHAR', 'message': 'JSON'})"),
}


def lot_du(fichier: str, regle: str) -> str:
    chemin = Path(fichier)
    if regle == "dossier":
        return chemin.parent.name
    if regle == "jour":
        return chemin.stem.replace("sorties_", "")
    return "complet"


def empreinte(fichier: str) -> str:
    return hashlib.sha256(Path(fichier).read_bytes()).hexdigest()


def espacer(nombre: int) -> str:
    return f"{nombre:,}".replace(",", " ")


def ouvrir_entrepot():
    Path(ENTREPOT).parent.mkdir(parents=True, exist_ok=True)
    try:
        con = duckdb.connect(ENTREPOT)
    except duckdb.IOException as erreur:
        sys.exit(f"L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.\n{erreur}")
    con.execute("CREATE SCHEMA IF NOT EXISTS raw")
    con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
    con.execute("""
        CREATE TABLE IF NOT EXISTS pipeline.chargements (
            fichier   VARCHAR,       -- le chemin du fichier, depuis Projet_Boreal
            empreinte VARCHAR,       -- SHA-256 du contenu
            table_raw VARCHAR,
            lot       VARCHAR,
            lignes    BIGINT,        -- lignes chargées ; 0 pour un fichier vide
            charge_le TIMESTAMPTZ)""")
```


© 2026 iAlexMG. Tous droits réservés.

```
    return con


def charger(con, table: str, motif: str, forme: str, regle: str) -> tuple:
    """Charge les fichiers nouveaux ou modifiés d'une livraison. Rend (fichiers, lignes)."""
    fichiers = sorted(p.as_posix() for p in Path(".").glob(motif))
    connus = dict(con.execute("""
        SELECT fichier, empreinte FROM pipeline.chargements WHERE table_raw = ?
        QUALIFY ROW_NUMBER() OVER (PARTITION BY fichier ORDER BY charge_le DESC) = 1""",
                              [table]).fetchall())
    a_charger = [(f, e, lot_du(f, regle)) for f in fichiers if connus.get(f) != (e := empreinte(f))]
    if not a_charger:
        return 0, 0

    source = LECTEURS[forme]
    noms = [f for f, _, _ in a_charger]
    charge_le = datetime.now(timezone.utc)
    con.execute("BEGIN")
    try:
        # La table naît de la forme du premier fichier : ses colonnes, en texte, puis le lignage.
        con.execute(f"""
            CREATE TABLE IF NOT EXISTS raw.{table} AS
            SELECT * EXCLUDE (filename), filename AS _fichier, '' AS _lot, NOW() AS _charge_le
            FROM ({source}) LIMIT 0""", {"fichiers": noms[:1]})
        con.execute("CREATE OR REPLACE TEMP TABLE a_charger (fichier VARCHAR, empreinte VARCHAR, lot VARCHAR)")
        con.executemany("INSERT INTO a_charger VALUES (?, ?, ?)", a_charger)
        # Un fichier déjà chargé sous une autre empreinte : ses anciennes lignes partent d'abord.
        con.execute(f"DELETE FROM raw.{table} WHERE _fichier IN (SELECT fichier FROM a_charger)")
        con.execute(f"""
            INSERT INTO raw.{table}
            SELECT s.* EXCLUDE (filename), s.filename, a.lot, $charge_le
            FROM ({source}) s JOIN a_charger a ON a.fichier = s.filename""",
                    {"fichiers": noms, "charge_le": charge_le})
        con.execute(f"""
            INSERT INTO pipeline.chargements
            SELECT a.fichier, a.empreinte, '{table}', a.lot, COUNT(r._fichier), $charge_le
            FROM a_charger a LEFT JOIN raw.{table} r
              ON r._fichier = a.fichier AND r._charge_le = $charge_le
            GROUP BY ALL""", {"charge_le": charge_le})
        lignes = con.execute(f"SELECT COUNT(*) FROM raw.{table} WHERE _charge_le = ?", [charge_le]).fetchone()[0]
        con.execute("COMMIT")
    except Exception:
```


© 2026 iAlexMG. Tous droits réservés.

```
            con.execute("ROLLBACK")      # qui ouvre la transaction la referme, même quand ça rate
            raise
        return len(a_charger), lignes


    def main():
        os.chdir(PROJET)
        con = ouvrir_entrepot()
        debut = time.perf_counter()
        total_fichiers = total_lignes = 0
        for table, motif, forme, regle in LIVRAISONS:
            fichiers, lignes = charger(con, table, motif, forme, regle)
            if fichiers:
                print(f"{table:26} {fichiers:4} fichier(s) chargé(s), {espacer(lignes):>8} lignes")
            else:
                print(f"{table:26}    à jour")
            total_fichiers += fichiers
            total_lignes += lignes
        con.close()
        print(f"Total : {espacer(total_fichiers)} fichier(s), {espacer(total_lignes)} lignes, "
              f"en {time.perf_counter() - debut:.1f} s.")


    if __name__ == "__main__":
        sys.stdout.reconfigure(encoding="utf-8", errors="replace")
        main()
```

2.2 Erreur voulue : lancer le chargeur pendant que l'interface tient l'entrepôt.

```
python 01_de\ingestion\charger_raw.py
```

Attendu : une erreur, `L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.`, suivie de `IO Error: Cannot open file "…\boreal.duckdb": Le processus ne peut pas accéder au fichier car ce fichier est utilisé par un autre processus.` et du programme qui le tient, `File is already open in …\duckdb.exe`. Rien n'est chargé.

À retenir — un seul programme à la fois sur l'entrepôt. Sous Windows, le programme qui tient `boreal.duckdb` ouvert en bloque tout autre, même quand il ne fait que lire : ici, l'interface, qui l'a attaché en 1.2. Le message du chargeur nomme la ligne de commande DuckDB ; l'interface web compte tout autant. On le provoque ici pour le reconnaître : chaque cours qui écrit dans l'entrepôt peut le rencontrer, et l'orchestration du Cours 06 doit en tenir compte.

2.3 Détacher l'entrepôt de l'interface, dans le carnet.

```
-- Attendu : 0 rows returned. entrepot disparaît de la liste Attached databases.
DETACH entrepot;
```


© 2026 iAlexMG. Tous droits réservés.

2.4 Relancer le chargeur. Première Anaconda Prompt, toujours dans `Projet_Boreal` :

```
python 01_de\ingestion\charger_raw.py
```

Attendu : 22 lignes, une par livraison, de `erp_t005u 49 fichier(s) chargé(s), 980 lignes` à `corpus_index 1 fichier(s) chargé(s), 33 lignes`, puis `Total : 1 572 fichier(s), 513 857 lignes`. Chez l'auteur, environ 12 s ; la durée dépend du poste, les comptes non.

2.5 Relancer le chargeur, pour prouver qu'il ne double rien.

```
python 01_de\ingestion\charger_raw.py
```

Attendu : `à jour` sur les 22 lignes, puis `Total : 0 fichier(s), 0 lignes`, en moins d'une seconde. C'est l'idempotence qui manquait en 1.4 : le journal a reconnu chaque fichier.

Étape 3 — Retrouver dans la RAW ce qu'elle garde exprès

Théorie : `Theorie_03_DE.md`, annexe A.3.

Dans le carnet.

3.1 Rattacher l'entrepôt, en lecture seule cette fois : on regarde, on ne touche à rien.

```
-- Attendu : 0 rows returned.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
```

3.2 Compter les tables de la `RAW`.

```
-- Attendu : 22.
SELECT COUNT(*) AS tables_raw
FROM information_schema.tables
WHERE table_catalog = 'entrepot' AND table_schema = 'raw';
```

3.3 Résumer le journal, livraison par livraison.

```
-- Attendu : 22 lignes, qui font 1 572 fichiers et 513 857 lignes. Des fichiers vides : erp_cdhdr et
-- erp_cdpos (44 chacun), garanties_reclamations et garanties_visites (1 chacun), telemetrie_sorties (41).
SELECT table_raw, COUNT(*) AS fichiers, SUM(lignes) AS lignes,
       COUNT(*) FILTER (WHERE lignes = 0) AS vides
FROM entrepot.pipeline.chargements
GROUP BY table_raw
ORDER BY table_raw;
```

3.4 Voir la forme d'une table de la `RAW`.

© 2026 iAlexMG. Tous droits réservés.

```
-- Attendu : 15 lignes. Les 12 colonnes de l'extrait, de MANDT à WAERK, en VARCHAR ; puis _fichier
-- VARCHAR, _lot VARCHAR et _charge_le TIMESTAMP WITH TIME ZONE.
DESCRIBE entrepot.raw.erp_vbak;
```

3.5 Compter les en-têtes des deux livraisons de mars 2023.

```
-- Attendu : 2 lignes, 609 en-têtes au lot 2023-03 et 609 au lot 2023-03_bis.
SELECT _lot, COUNT(*) AS en_tetes
FROM entrepot.raw.erp_vbak
WHERE _lot LIKE '2023-03%'
GROUP BY _lot
ORDER BY _lot;
```

3.6 Comparer les lignes de `erp_vbak` aux commandes.

```
-- Attendu : 26 538 lignes pour 25 483 commandes.
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes
FROM entrepot.raw.erp_vbak;
```

3.7 Compter les messages de télémétrie et leurs identifiants.

```
-- Attendu : 158 146 messages pour 155 843 identifiants.
SELECT COUNT(*) AS messages, COUNT(DISTINCT message ->> 'message_id') AS identifiants
FROM entrepot.raw.telemetrie_sorties;
```

3.8 Compter les versions des réclamations.

```
-- Attendu : 5 623 objets pour 3 348 réclamations.
SELECT COUNT(*) AS objets, COUNT(DISTINCT objet ->> 'reclamation_id') AS reclamations
FROM entrepot.raw.garanties_reclamations;
```

3.9 Suivre le client effacé d'un lot à l'autre, sans afficher ses données.

```
-- Attendu : 2 lignes. Lot 2024-01 : prénom présent, ACTIF ; lot 2025-09 : prénom absent, EFFACE.
SELECT _lot, prenom IS NOT NULL AS prenom_present, statut
FROM entrepot.raw.garanties_clients
WHERE client_id = 'CL0011163'
ORDER BY _lot;
```

Étape 4 — Déposer le corpus tel quel, avec son catalogue

Théorie : `Theorie_03_DE.md`, annexe A.4.

4.1 Créer le script. Première Anaconda Prompt, depuis `Projet_Boreal` :

© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\ingestion\deposer_corpus.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""deposer_corpus.py : la documentation de Boréal déposée telle quelle dans le bronze.

Lancement, depuis Projet_Boreal/01_de : python ingestion/deposer_corpus.py
"""
import hashlib
import shutil
import sys
from pathlib import Path

PROJET = Path(__file__).resolve().parents[2]
SOURCE = PROJET / "sources" / "corpus"
BRONZE = PROJET / "plateforme" / "lac" / "bronze" / "corpus"


def empreinte(chemin: Path) -> str:
    return hashlib.sha256(chemin.read_bytes()).hexdigest()


def main():
    copies = identiques = 0
    for fichier in sorted(p for p in SOURCE.rglob("*") if p.is_file()):
        cible = BRONZE / fichier.relative_to(SOURCE)
        if cible.exists() and empreinte(cible) == empreinte(fichier):
            identiques += 1
            continue
        cible.parent.mkdir(parents=True, exist_ok=True)
        shutil.copyfile(fichier, cible)
        copies += 1
    print(f"Corpus : {copies} fichier(s) déposé(s), {identiques} déjà à jour, dans "
          f"{BRONZE.relative_to(PROJET).as_posix()}")


if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    main()
```

4.2 Lancer le dépôt. Ce script ne touche pas à l'entrepôt : il tourne même quand l'interface le tient.

© 2026 iAlexMG. Tous droits réservés.

```
python 01_de\ingestion\deposer_corpus.py
```

Attendu : `Corpus : 35 fichier(s) déposé(s), 0 déjà à jour, dans plateforme/lac/bronze/corpus.`

4.3 Relancer le dépôt, pour prouver qu'il ne recopie rien.

```
python 01_de\ingestion\deposer_corpus.py
```

Attendu : `Corpus : 0 fichier(s) déposé(s), 35 déjà à jour, dans plateforme/lac/bronze/corpus.`

4.4 Peser le corpus déposé, dans le carnet.

```
-- Attendu : 35 fichiers, 135 714 octets.
SELECT COUNT(*) AS fichiers, SUM(size) AS octets
FROM read_blob('plateforme/lac/bronze/corpus/**');
```

4.5 Compter les documents du catalogue dans la `RAW`.

```
-- Attendu : 33.
SELECT COUNT(*) AS documents
FROM entrepot.raw.corpus_index;
```

4.6 Détacher l'entrepôt : le Cours 04 écrira dedans.

```
-- Attendu : 0 rows returned.
DETACH entrepot;
```

Étape 5 — Versionner le chargeur et le dépôt du corpus

Théorie : `Theorie_03_DE.md`, annexe A.5.

Première Anaconda Prompt, depuis `Projet_Boreal` :

```
git -C 01_de add ingestion\charger_raw.py ingestion\deposer_corpus.py
git -C 01_de commit -m "Cours 03 : couche RAW et corpus déposé"
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `.exit` à l'invite `D` de la seconde Anaconda Prompt, puis fermer l'onglet du navigateur.

Dépannage

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Correctif
`IO Error: Cannot open file "…\boreal.duckdb": Le chemin d'accès spécifié est introuvable.` en 1.2	le `mkdir` de 1.1 n'a pas été fait : le faire, puis relancer la cellule
`L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.` hors de 2.2	l'interface tient encore l'entrepôt : `DETACH entrepot;` dans le carnet (ou `.exit` dans la seconde Anaconda Prompt), puis relancer le script
2.2 charge tout au lieu de refuser	l'interface ne tenait plus l'entrepôt (fermée ou relancée depuis 1.2) : le chargement est bon ; passer à 2.5, qui doit répondre `à jour` partout
`Binder Error: Catalog "entrepot" does not exist!`, ou 3.2 qui rend 0	l'entrepôt n'est pas attaché (interface relancée, ou `DETACH` déjà fait) : relancer la cellule de 3.1
`Binder Error: Failed to detach database with name "entrepot": database not found`	l'entrepôt était déjà détaché : rien à faire, passer à la suite
`python: can't open file` sur `charger_raw.py` ou `deposer_corpus.py`	la fenêtre n'est pas dans `Projet_Boreal`, ou le Bloc-notes a enregistré le script en `.py.txt` (`dir 01_de\ingestion` le montre) : revenir dans `Projet_Boreal`, ou `ren` pour retirer le `.txt`
`ModuleNotFoundError: No module named 'duckdb'`	l'environnement `formations` n'est pas actif : `conda activate formations`, puis relancer
La cellule n'affiche qu'un seul résultat	c'est voulu en 1.3 et 1.4 : la cellule montre sa dernière requête, le contrôle
La page ne répond plus	la seconde Anaconda Prompt a été fermée : y relancer `duckdb -ui` depuis `Projet_Boreal`, puis l'`ATTACH` de 3.1
Un compte ne donne pas le chiffre attendu	les sources ont changé depuis le chargement : `DETACH entrepot;`, effacer `plateforme\entrepot\boreal.duckdb`, relancer le générateur depuis `Data_Domain` (`python _univers/boreal/generer_boreal.py --projet Projet_Boreal`), les deux scripts du Labo 02, puis le chargeur

---

Annexe — Sur Snowflake

Pour qui charge la `RAW` dans Snowflake plutôt que dans DuckDB. Les faits ont été vérifiés dans la documentation de Snowflake en septembre 2026 ; les commandes se tapent dans une feuille de calcul de Snowsight, sur votre propre compte. L'exemple charge `VBAK` ; chaque table suit le même patron.

© 2026 iAlexMG. Tous droits réservés.

Snowflake calcule dans le nuage : ses entrepôts ne voient pas les fichiers de votre disque. Un fichier doit donc d'abord entrer dans un stage, un espace de stockage que Snowflake connaît. Un stage interne est géré par Snowflake ; un stage externe pointe vers votre propre stockage objet (Amazon S3, Azure Blob Storage, Google Cloud Storage). En local, la question ne se pose pas : DuckDB lit les fichiers là où ils sont. Un stage n'a rien à voir avec la couche `STAGING` d'Analytics Engineering, malgré le mot.

```
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS BOREAL;
CREATE TRANSIENT SCHEMA IF NOT EXISTS BOREAL.RAW;
CREATE STAGE IF NOT EXISTS BOREAL.RAW.LIVRAISONS;
```

Pourquoi un schéma transient. Snowflake protège une table ordinaire deux fois : le voyage dans le temps (*Time Travel*), qui garde les versions passées pendant la durée choisie, puis sept jours de sécurité intégrée (*Fail-safe*), réservés à la reprise après sinistre par Snowflake lui-même, et facturés en stockage. Une table transient n'a pas de sécurité intégrée, et son voyage dans le temps dure 0 ou 1 jour. Toute table créée dans un schéma transient est transient. La `RAW` se reconstruit à partir des fichiers livrés, qui sont sa vraie copie de sûreté : payer la sécurité intégrée pour elle n'achète rien. Une couche qui est le seul exemplaire d'un travail, elle, garde la protection complète.

Pourquoi `SYSADMIN`, et pas un rôle plus fort. Dans Snowflake, le rôle qui crée un objet en devient propriétaire. Créer la base et le schéma avec `SYSADMIN`, le rôle prévu pour les objets de données, évite qu'ils appartiennent à `ACCOUNTADMIN`, que les autres rôles ne voient pas. En équipe, le chargement passe ensuite par un rôle dédié, qui n'a de droits que sur `RAW` : une erreur de script ne peut rien abîmer ailleurs.

Déposez les fichiers dans le stage, par Snowsight : menu Ingestion, Add Data, Load files into a Stage. Choisissez le schéma `RAW` et le stage `LIVRAISONS`, et précisez un chemin qui garde le mois, `erp/extraits/2023-03/` par exemple : le lot se lira dans ce chemin. Un fichier y est limité à 250 Mo. Un client en ligne de commande (Snowflake CLI) fait le même dépôt par la commande `PUT`.

```
CREATE TABLE IF NOT EXISTS BOREAL.RAW.ERP_VBAK (
    MANDT VARCHAR, VBELN VARCHAR, ERDAT VARCHAR, ERZET VARCHAR, AEDAT VARCHAR, AUART VARCHAR,
    VKORG VARCHAR, VTWEG VARCHAR, KUNNR VARCHAR, VDATU VARCHAR, NETWR VARCHAR, WAERK VARCHAR,
    _FICHIER VARCHAR, _LOT VARCHAR, _CHARGE_LE TIMESTAMP_LTZ);

COPY INTO BOREAL.RAW.ERP_VBAK
FROM (
    SELECT $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12,
           METADATA$FILENAME, SPLIT_PART(METADATA$FILENAME, '/', -2), METADATA$START_SCAN_TIME
    FROM @BOREAL.RAW.LIVRAISONS)
PATTERN = '.*extraits/.*/VBAK[.]csv'
FILE_FORMAT = (TYPE = CSV SKIP_HEADER = 1 FIELD_OPTIONALLY_ENCLOSED_BY = '"');
```

Élément	Ce qu'il fait
`$1` … `$12`	les colonnes du CSV, par position

© 2026 iAlexMG. Tous droits réservés.

Élément	Ce qu'il fait
`METADATA$FILENAME`	le chemin du fichier dans le stage : le `_fichier` du labo
`SPLIT_PART(…, '/', -2)`	l'avant-dernier morceau du chemin, le dossier du mois : le `_lot` ; un rang négatif compte depuis la fin
`METADATA$START_SCAN_TIME`	l'heure du chargement : le `_charge_le`
`PATTERN`	une expression régulière sur le chemin : seulement les `VBAK.csv` des extraits
`SKIP_HEADER = 1`	saute l'en-tête : la valeur par défaut est 0

`COPY INTO` tient son propre journal. Pendant 64 jours, la table garde le nom, la taille et l'empreinte de chaque fichier chargé, et refuse de le recharger : relancer la commande ne double rien. L'option `FORCE = TRUE` passe outre et double les lignes ; `LOAD_UNCERTAIN_FILES = TRUE` charge les fichiers dont l'historique a expiré. Comme le journal du labo, celui de Snowflake laisse passer `2023-03_bis`, qui porte un autre nom. Un champ vide devient `NULL` par défaut (`EMPTY_FIELD_AS_NULL`), comme dans la `RAW` locale.

Snowpipe charge les fichiers au fil de leur arrivée, sans qu'on relance `COPY INTO`. Il apprend l'arrivée d'un fichier par les notifications d'événements d'un stockage objet, ou par un appel de ses points d'accès REST, et garde son historique de chargement 14 jours. La formation Snowflake le traite en détail.

Annexe — Sur Databricks

Pour qui charge la `RAW` dans Databricks. Les faits ont été vérifiés dans la documentation de Databricks en septembre 2026. Databricks Free Edition suffit : calcul serverless seulement, un entrepôt SQL de taille 2X-Small, des quotas qui coupent le calcul pour le reste de la journée quand ils sont dépassés, et aucun usage commercial. Serverless veut dire que la plateforme fournit le calcul et le facture à l'usage, sans grappe de machines à dimensionner. Les commandes se tapent dans l'éditeur SQL, sur votre espace.

Les fichiers entrent dans un volume Unity Catalog, un espace de fichiers rangé dans un schéma, que les requêtes lisent par un chemin `/Volumes/<catalogue>/<schéma>/<volume>/`.

```
CREATE SCHEMA IF NOT EXISTS raw;
CREATE VOLUME IF NOT EXISTS raw.livraisons;
```

Déposez les dossiers `erp/extraits/…` dans le volume : menu New, Add or upload data, Upload files to a volume. Un fichier y est limité à 5 Go. Créez ensuite la table, vide et sans schéma, puis chargez :

```
CREATE TABLE IF NOT EXISTS raw.erp_vbak;

COPY INTO raw.erp_vbak
```


© 2026 iAlexMG. Tous droits réservés.

```
FROM (
    SELECT *, _metadata.file_path AS _fichier,
           element_at(split(_metadata.file_path, '/'), -2) AS _lot,
           current_timestamp() AS _charge_le
    FROM '/Volumes/<catalogue>/raw/livraisons/erp/extraits/')
FILEFORMAT = CSV
PATTERN = '*/VBAK.csv'
FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'false')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Élément	Ce qu'il fait
`_metadata.file_path`	le chemin du fichier lu, par la colonne cachée `_metadata` : le `_fichier` du labo
`element_at(split(…), -2)`	l'avant-dernier morceau du chemin, le dossier du mois : le `_lot`
`'inferSchema' = 'false'`	aucune détection de types : toutes les colonnes arrivent en texte
`'mergeSchema' = 'true'`	la table vide prend la forme du premier chargement
`PATTERN`	un motif de noms de fichiers, relatif au chemin du `FROM`

`COPY INTO` est idempotent : il saute les fichiers déjà chargés, même s'ils ont été modifiés depuis. C'est la différence avec le chargeur du labo, qui recharge un fichier corrigé sous le même nom ; ici, il faut l'option `'force' = 'true'`, qui recharge tout et double les lignes. Pour un chargement continu, Auto Loader, une source de Structured Streaming nommée `cloudFiles`, suit les fichiers déjà traités dans son point de reprise et garantit que chacun n'est traité qu'une fois ; il lit aussi les volumes Unity Catalog. Pour les utilisateurs de SQL, Databricks recommande désormais les tables de diffusion en continu (*streaming tables*), qui reposent sur ce mécanisme.

© 2026 iAlexMG. Tous droits réservés.